NoSQL Databases

MSc Course @ University of Insubria

Roberto Vicario

2025/2026

Contents

1 Introduction **2**

2 Key-Value Stores **2**

3 Redis **2**

3.1 Data Model 2
3.2 Data Persistence 2
3.3 Data Structures 2
3.4 Strings 3
3.4.1 Commands (CLI) 4
3.5 Hashes 6
3.5.1 Commands (CLI) 7
3.6 Lists 12
3.6.1 Commands (CLI) 13
3.7 Sets 13
3.7.1 Commands (CLI) 14
3.8 Sorted Sets 15
3.8.1 Commands (CLI) 16
3.9 Advanced Features 17
3.9.1 Transactions 17
3.9.2 Secondary Indexes 18
3.9.3 Composite Indexes 18

1 Introduction

…

2 Key-Value Stores

…

3 Redis

3.1 Data Model

Redis is a ***key-value*** store. It stores data as a collection of key-value pairs, where:

- **Key**: A unique identifier for each data item.
- **Value**: The actual data associated with the key. Values could be categorized into ***primitives*** (strings) and **containers** (complex data structures).

3.2 Data Persistence

Redis guarantees this property by using the following components:

- **Redis Instance**: A ***main memory database*** that holds data and serves client requests.
- **RDB Snapshots**: Periodic point-in-time dumps of the dataset to disk in ***RDB (Redis Database)*** format.
- **AOF Log**: An ***append-only file*** that records every write operation, enabling dataset reconstruction after crashes.

3.3 Data Structures

Redis is a ***data structure server***. At its core, Redis provides a collection of ***native data types*** that help you solve a wide variety of problems, from caching to queuing to event processing.

Redis implements the following data types:

- ***String***
- ***Hash***

- ***List***
- ***Set***
- ***Sorted set***
- Vector set
- Stream
- Bitmap
- Bitfield
- Geospatial
- JSON
- Probabilistic data types
- Time series

3.4 Strings

Strings is an ***array*** data structure ***of bytes*** (or words) that stores a sequence of elements, typically characters, using some character encoding.

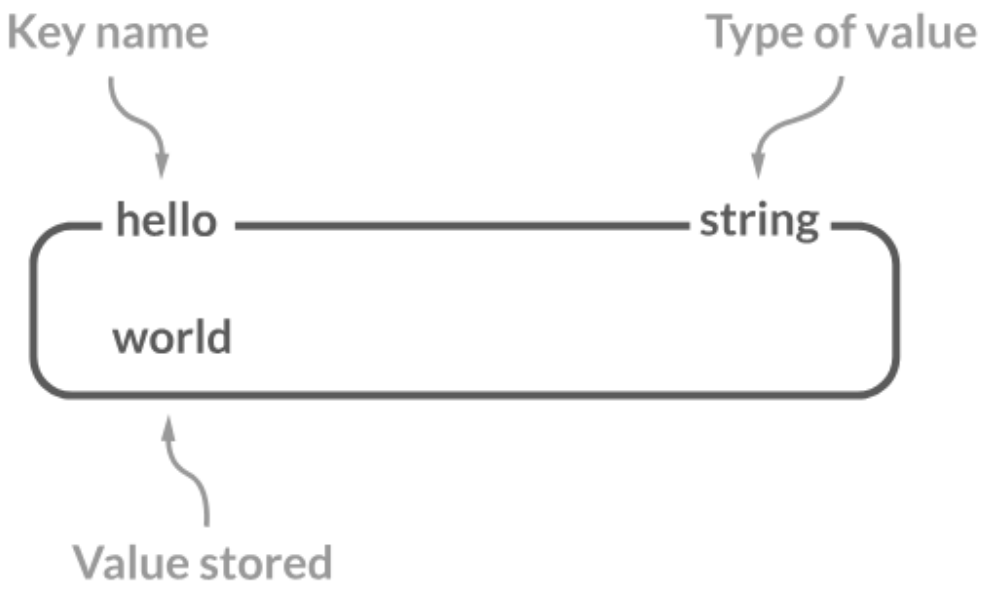

Figure 1: Strings Data Structure

3.4.1 Commands (CLI)

The main commands for the strings are:

- `GET`: Get the value of a key.
- `SET`: Set the value of a key.
- `EXISTS`: Check if a key exists.
- `DEL`: Delete a key.

> **Time Complexity:** All the commands above have a time complexity of $O(1)$, where 1 is the number (N) of operations performed.

GET

```
# --- Syntax ---
GET key
```

```
# --- Examples ---
redis> GET nonexisting
(nil)
redis> SET mykey "Hello"
"OK"
redis> GET mykey
"Hello"
```

Figure 2: `GET` – Syntax Diagram

SET

```
# --- Syntax ---
SET key value
    [NX | XX | IFEQ ifeq-value | IFNE ifne-value | IFDEQ ifdeq-digest |
        IFDNE ifdne-digest]
    [GET]
    [EX seconds | PX milliseconds | EXAT unix-time-seconds | PXAT unix-
       time-milliseconds | KEEPTTL]
```

```redis
# --- Examples ---
redis> SET mykey "Hello"
"OK"
redis> GET mykey
"Hello"
redis> SET anotherkey "will expire in a minute" EX 60
"OK"
```

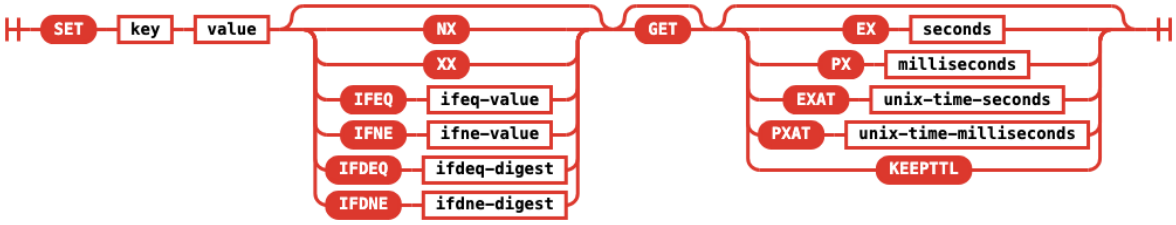

Figure 3: SET – Syntax Diagram

EXISTS

```redis
# --- Syntax ---
EXISTS key [key ...]
```

```redis
# --- Examples ---
redis> SET key1 "Hello"
"OK"
redis> EXISTS key1
(integer) 1
redis> EXISTS nosuchkey
(integer) 0
redis> SET key2 "World"
"OK"
redis> EXISTS key1 key2 nosuchkey
(integer) 2
```

Figure 4: EXISTS – Syntax Diagram

DEL

```
# --- Syntax ---
DEL key [key ...]
```

```
# --- Examples ---
DEL key [key ...]
```

Figure 5: DEL – Syntax Diagram

3.5 Hashes

An hash is a data type that represents a mapping between a string ***field*** and a string ***value***.

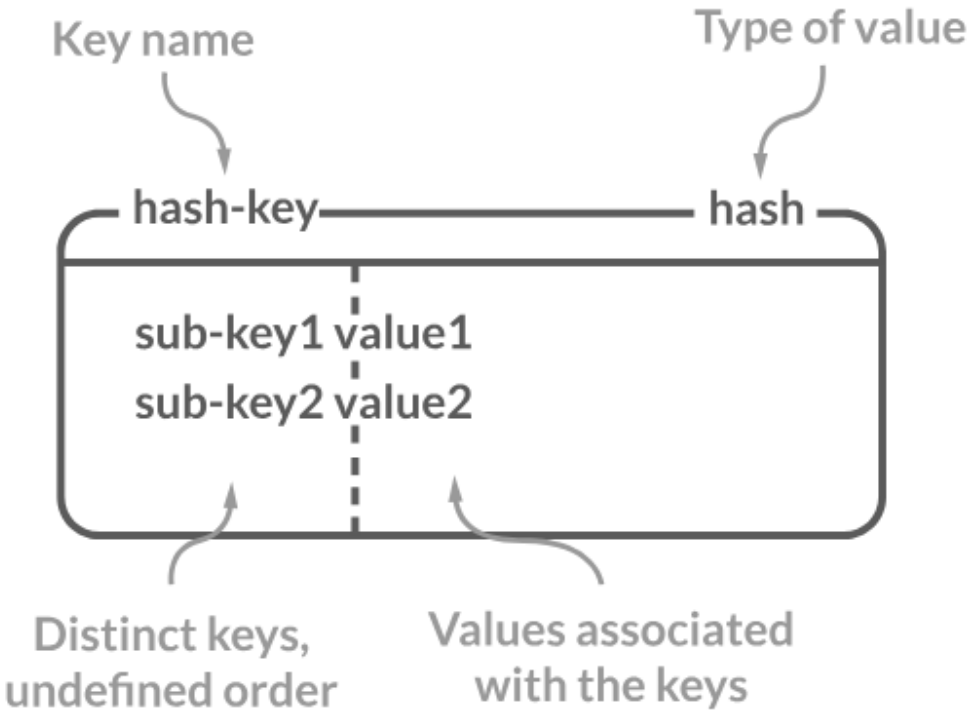

Figure 6: Hashes Data Structure

3.5.1 Commands (CLI)

The main commands for hashes are:

- `HGET`: Get the value of a field in a hash.
- `HSET`: Set the value of a field in a hash.
- `HEXISTS`: Check if a field exists in a hash.
- `HDEL`: Delete a field from a hash.

> **Time Complexity:** All the commands above have a time complexity of $O(1)$. The following ones have a time complexity of $O(N)$, where N is the number of operations performed, due to multiple (M) returns.

- `HMGET`: Get the value of a field in a hash.
- `HMSET`: Set the value of a field in a hash.
- `HKEYS`: Get all the keys in a hash.

- HVALS: Get all the values in a hash.

HGET

```
# --- Syntax ---
HGET key field
```

```
# --- Examples ---
redis> HSET myhash field1 "foo"
(integer) 1
redis> HGET myhash field1
"foo"
redis> HGET myhash field2
(nil)
```

Figure 7: HGET – Syntax Diagram

HSET

```
# --- Syntax ---
HSET key field value [field value ...]
```

```
# --- Examples ---
redis> HSET myhash field1 "Hello"
(integer) 1
redis> HGET myhash field1
"Hello"
redis> HSET myhash field2 "Hi" field3 "World"
(integer) 2
redis> HGET myhash field2
"Hi"
redis> HGET myhash field3
"World"
redis> HGETALL myhash
1) "field1"
2) "Hello"
3) "field2"
4) "Hi"
5) "field3"
6) "World"
```

Figure 8: HSET – Syntax Diagram

HEXISTS

```
# --- Syntax ---
HEXISTS key field
```

```
# --- Examples ---
redis> HSET myhash field1 "foo"
(integer) 1
redis> HEXISTS myhash field1
(integer) 1
redis> HEXISTS myhash field2
(integer) 0
```

Figure 9: HEXISTS – Syntax Diagram

HDEL

```
# --- Syntax ---
HDEL key field [field ...]
```

```
# --- Examples ---
redis> HSET myhash field1 "foo"
(integer) 1
redis> HDEL myhash field1
(integer) 1
redis> HDEL myhash field2
(integer) 0
```

Figure 10: HDEL – Syntax Diagram

HMGET

```
# --- Syntax ---
HMGET key field [field ...]
```

```
# --- Examples ---
redis> HSET myhash field1 "Hello"
(integer) 1
redis> HSET myhash field2 "World"
(integer) 1
redis> HMGET myhash field1 field2 nofield
1) "Hello"
2) "World"
3) (nil)
```

Figure 11: HMGET – Syntax Diagram

HMSET

```
# --- Syntax ---
HMSET key field value [field value ...]
```

```
# --- Examples ---
redis> HMSET myhash field1 "Hello" field2 "World"
"OK"
redis> HGET myhash field1
"Hello"
redis> HGET myhash field2
"World"
```

Figure 12: HMSET – Syntax Diagram

HKEYS

```
# --- Syntax ---
HKEYS key
```

```
# --- Examples ---
redis> HSET myhash field1 "Hello"
(integer) 1
redis> HSET myhash field2 "World"
(integer) 1
redis> HKEYS myhash
1) "field1"
2) "field2"
```

Figure 13: HKEYS – Syntax Diagram

HVALS

```
# --- Syntax ---
HVALS key
```

```
# --- Examples ---
redis> HSET myhash field1 "Hello"
(integer) 1
redis> HSET myhash field2 "World"
(integer) 1
```

```
redis> HVALS myhash
1) "Hello"
2) "World"
```

Figure 14: HVALS – Syntax Diagram

3.6 Lists

Lists holds collections of ***string*** elements sorted according to their ***order of insertion***.

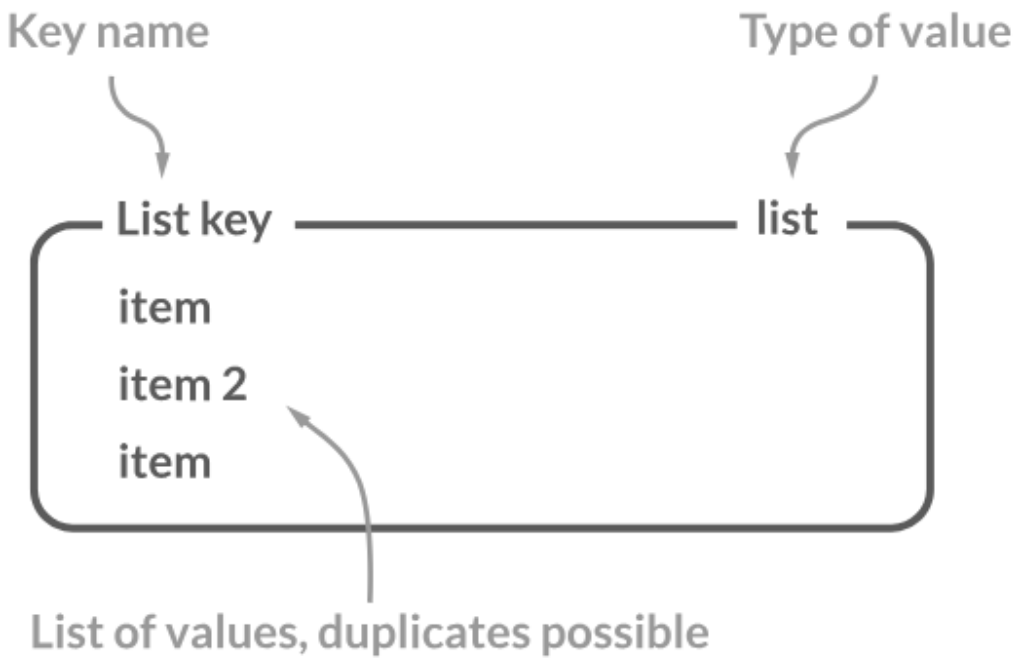

Figure 15: Lists Data Structure

3.6.1 Commands (CLI)

The main commands for the strings are:

- `LLEN`: Get the length of a list.
- `LPUSH`: Add an element to the beginning of a list.
- `RPUSH`: Add an element to the end of a list.
- `LPOP`: Remove and return the first element of a list.
- `RPOP`: Remove and return the last element of a list.

> **Time Complexity:** All the commands above have a time complexity of $O(1)$. The following ones have a time complexity of $O(N)$.

- `LINDEX`: Get the element at the specified index in a list.
- `LSET`: Set the value of an element in a list by its index.
- `LINSERT`: Insert an element before or after another element in a list.
- `LRANGE`: Get a range of elements from a list.

LLEN

3.7 Sets

Sorted sets contain a unique set of ***members ordered by*** floating-point ***scores***.

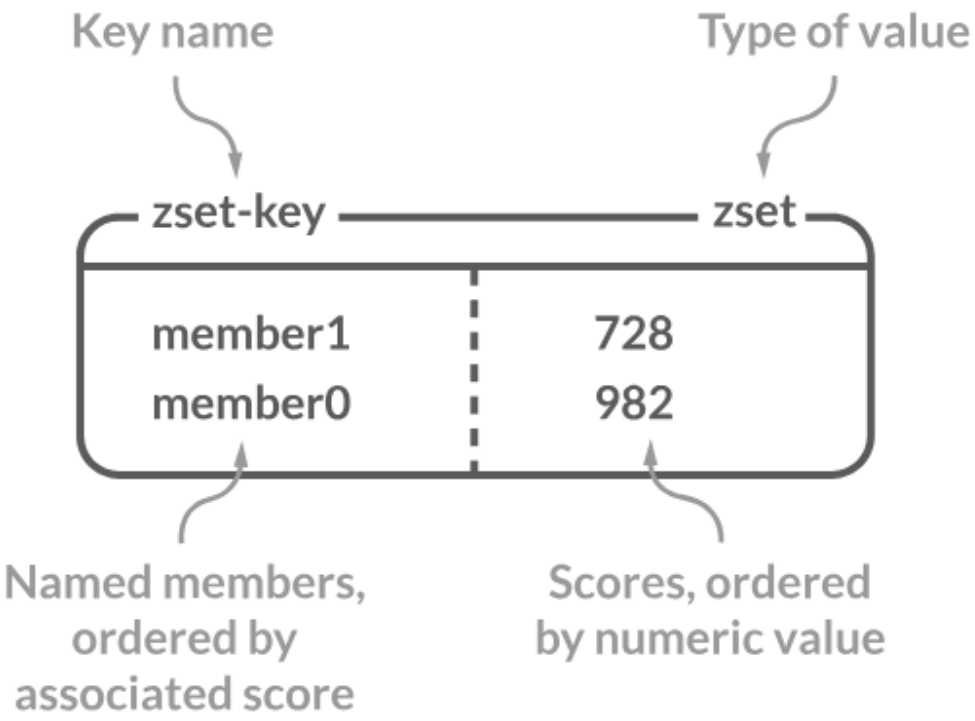

Figure 16: Sets Data Structure

3.7.1 Commands (CLI)

The main commands for the sets are:

- `SADD`: Add a member to a set.
- `SREM`: Remove a member from a set.
- `SCARD`: Get the number of members in a set.

> **Time Complexity:** All the commands above have a time complexity of $O(1)$. The following ones have a time complexity of $O(N)$.

- `SDIFF`: Get the difference between two or more sets.
- `SUNION`: Get the union of two or more sets.
- `SMEMBERS`: Get all the members in a set.

> **Time Complexity:** The following command have a time complexity of $O(C * M)$, where C is the number of sets involved and M is the number of members returned.

- `SINTER`: Get the intersection of two or more sets.

SADD

SREM

SCARD

SDIFF

SUNION

SMEMBERS

SINTER

3.8 Sorted Sets

Sorted sets contain a unique set of ***members ordered by*** floating-point scores_**.

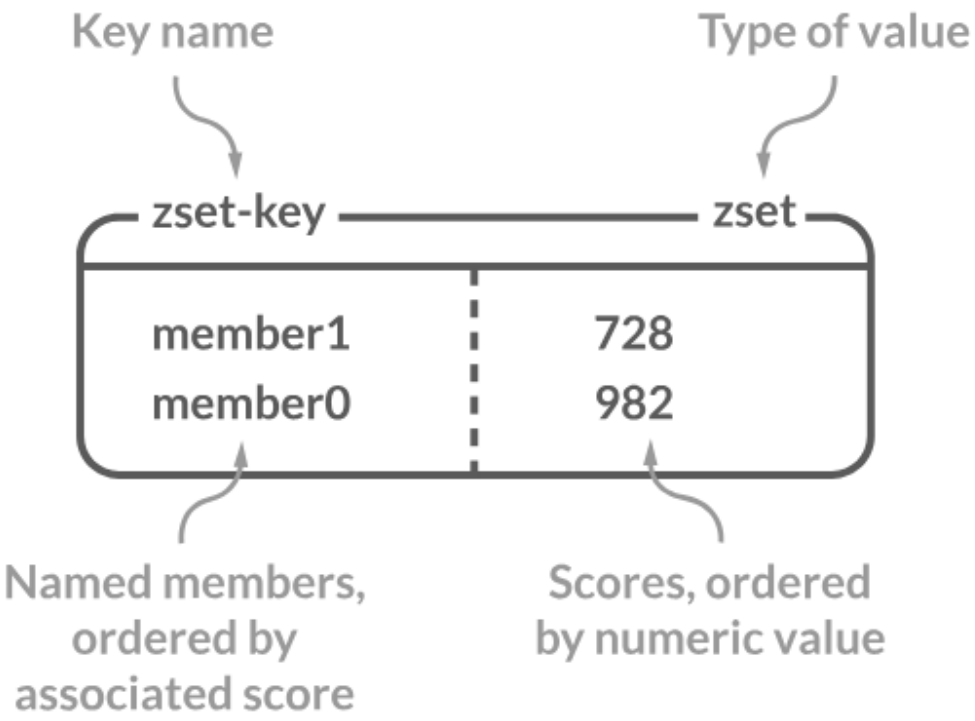

Figure 17: Sorted Sets Data Structure

3.8.1 Commands (CLI)

The main commands for the strings are:

- ZCARD: Get the number of members in a sorted set. Its ***time complexity*** is $O(1)$.

> **Time Complexity:** All the commands above have a time complexity of $O(1)$. The following ones have a time complexity of $O(\log(N))$, where N is the number of operations performed, due to multiple returns.

- ZADD: Add a member to a sorted set, or update its score if it already exists.
- ZRANK: Get the rank of a member in a sorted set, with scores ordered from low to high.
- ZREM: Remove a member from a sorted set.

> **Time Complexity:** The following ones have a time complexity of $O(\log(N) + M)$.

- `ZRANGE`: Get a range of members in a sorted set, by index.
- `ZRANGEBYSCORE`: Get a range of members in a sorted set, by score.

ZADD

ZRANK

ZREM

ZRANGE

ZRANGEBYSCORE

3.9 Advanced Features

Redis provides a set of advanced features that allow you to build complex applications on top of it. These features include:

3.9.1 Transactions

Redis supports transactions to execute multiple commands ***atomically***:

- A transaction is ***started*** with `MULTI`, which queues commands, and ***executed*** with `EXEC`, which runs all queued operations sequentially.
- If needed, a transaction ***can be canceled*** using `DISCARD`.

Redis ***does not support rollback***: even if one command fails during execution, the others are still executed.

For concurrency control, `WATCH` can be used to implement ***optimistic locking*** by monitoring keys and aborting the transaction if they are modified before execution.

3.9.2 Secondary Indexes

Secondary indexes in Redis are used to efficiently query data ***based on attributes*** other than the primary key. They are typically implemented using ***sorted sets***, where the score represents the ***indexed attribute*** and the value is the object identifier.

3.9.3 Composite Indexes

Composite indexes allow querying based on multiple attributes ***simultaneously***. They are often implemented by encoding ***multiple fields into a single value*** (e.g., room:price:id) stored in a sorted set. Queries can then exploit ***lexicographical ordering*** or ***score-based filtering*** to efficiently retrieve matching elements.